pyaces Developer Manual

Structure, build, extension and testing of `aces2py`, the f2py extension module that drives ACES II from Python

Repository	`ACES_PROGRAM/pyaces` (GitHub youngchoonpark/ACES, private)
Reference commit	master, 2026-10-23 (TODO stage 1 applied)
Written	2026-10-23
Verified	pyaces suite 51/51 PASS on the local workstation (oneAPI 2024.0) and the ccsd server (oneAPI 2023.2); 60/60 on both after the 2026-10-30 reclassification and Runpccd fix
Link audit	296 collision-baseline entries, 0 COMMON shadows, identical on both machines

This manual was written from the source code (`pyaces/build.sh`, `a2_pyf/*.F`, the generated `Main/gen/aces2pymodule.c`, the test scripts) and from inspecting the built shared object. Line numbers refer to the reference commit.

Contents

1. Overview
2. Directory layout
3. Build environment
4. Build pipeline (`build.sh`)
5. Linking into one .so: symbol collisions and their fixes
6. The link-resolution audit (`collision_audit.py`)
7. The `aces2py` API
8. Running calculations
9. The test suite
10. Extending pyaces: new entry points and new tests
11. Known issues (with fix history)
12. Development history and server deployment

Appendix A. Legacy files · Appendix B. Command summary

1. Overview

pyaces lets Python call the computational modules of the Fortran quantum-chemistry program ACES II (`ACES_PROGRAM/aces2`) directly. Classic ACES II builds one executable per module (`xjoda`, `xvscf`, `xvcc`, ...); a driver runs them in turn and they exchange state through files on disk (JOBARC, MOINTS, ...). pyaces links those modules into **a single shared library**, `aces2py.cpython-312-x86_64-linux-gnu.so`, and puts a Python wrapper generated by numpy's f2py on top.

Python: `import aces2py; aces2py.Init(); aces2py.Runcc(0); aces2py.Dgetrecpy("TOTENERG", e)`

▼ f2py C wrapper (`Main/gen/aces2pymodule.c`)

pyaces Fortran shims: `Main/init.F`, `a2_pyf/run*.F`, `a2_pyf/*recpy.F`, `*lstpy.F`, `aux_pyaces/*.F`

▼ static link (`-Wl,--start-group` ... about 107 archives)

aces2 archives: `aces2/lib/lib*.a` (joda, vmol, vscf, vtran, intprc, vcc, lambda, vee, props, intgrt, dirmp2, ...), some replaced by patched copies in `build/patched_libs`

▼

Intel MKL ILP64 (sequential) and the Intel Fortran runtime

1.1 Key facts

- **The aces2 sources are never touched.** pyaces links aces2's `.a` files as they are. Only for archives with collisions does it make temporary copies and rename symbols in them with `objcopy`. The aces2 tree is shared with the classic driver and its test suite.
- **All integers are 64-bit.** aces2 and pyaces are compiled with `-i8`, and MKL is linked in its ILP64 flavour.
- **One calculation per process.** aces2's COMMON-block state assumes a single use; calling Vtran/Vcc twice in one process has been seen to converge silently to a wrong answer.
- **The main risk is at link time.** Routines that share a name but differ in implementation were harmless when every executable was linked on its own. In one .so they collide, and the linker silently picks one. The safeguards for this are the core of chapters 5 and 6.
- **Tests reuse the aces2 fast suite.** Of its 158 cases, the 60 that pyaces can run (SCF 9, CC 39, EOM 8, KS 3, PCCD 1) are executed and their JOBARC energies compared with the reference values to within 1×10^{-6} E_h.

1.2 How to read this manual

If you are new to pyaces, read chapter 2 (which files are live), then chapter 4 (the build), then chapters 7–8 (API and running). Before exposing a new Fortran routine or linking another aces2 module, read chapters 5, 6 and 10. Chapter 11 lists the defects still present in the code.

2. Directory layout

`pyaces/` still carries much of the original 2023 acespy, so it matters which files the current build actually uses. live marks files used by the build or tests, generated marks build output (gitignored), and legacy marks files nothing uses.

Path	Status	Contents
`build.sh`	live	The whole build (about 510 lines): f2py wrapper generation, C/Fortran compilation, patched archives, link, import test, link audit. The history of every collision fix is recorded in its comments.
`Main/init.F`, `Main/aces2_main.F`	live	The `Init()` entry point. Joda reads ZMAT, then `aces_init` sets up core memory, flags and lists.
`a2_pyf/*.F`	live	Wrappers and drivers exposed to Python: `run*.F`, `*recpy.F`, `*lstpy.F`, `getlstdimpy.F`, `*allpy.F`, `get1ehpy/get2ehpy.F`, `acesja*.F`, plus the internal `a2_get*/a2_put*.F`. `rungrads`, `run1props` and `run` are excluded from the build.
`aux_pyaces/*.F`	live	`init_legacies` (initialises legacy COMMON blocks), `get1ents` / `get2eints` (read AO integrals from the IIII file), `sym_expand`.
`tools/collision_audit.py`	live	The link-resolution audit (chapter 6), with `collision_baseline.txt` (296 entries) and `common_shadow_baseline.txt` (0).
`pyaces_run_case.py`	live	Runs one case in its own process and prints the result as marker lines (chapter 8).
`pyaces_suite_runner.py`	live	Runs the 60 cases and grades each one PASS/MISMATCH/... (chapter 9).
`classify_suite.py`, `suite_classification.tsv`	live	The table that classifies the aces2 fast-suite cases by category, entry point and reference value, and the script that generates it. **The TSV is generated; never edit it by hand** (section 9.4).
`docs/`	live	This manual (HTML source and PDF).
`Main/gen/`	generated	f2py output: `aces2pymodule.c`, `aces2py-f2pywrappers.f`.
`Main/build/`	generated	All `.o` files and the final `aces2py.cpython-312-x86_64-linux-gnu.so`.
`build/patched_libs/`	generated	Copies of aces2 archives with renamed symbols (about 37). Wiped and rebuilt on every build.
`f2py_gen.log`	generated	The f2py wrapper-generation log. Read it to see how the wrappers were built.
`tests/`	live	API tests: `api_cases.py` (the tests and their ZMATs) and `run_api_tests.py` (section 9.5).
`python/`	live	The `pyaces` Python package (`pyproject.toml`, `python/pyaces/`): lower-case in-process wrappers, a subprocess `run()`, and the MO-list tables in `pyaces.moio` (section 7.7).

`tools/localize.txt`	live	Every symbol rename and extracted module, one per line with its reason, plus the history of how each was found (section 5.2).
`README`, `frequently_asked_questions`, `LICENSE`	legacy	Old acespy build notes and the generic f2py FAQ; LICENSE (GPL v3) is still in force (Appendix A).

> **Removed legacy tree**: the acespy-era files (`Fxx/` , `include/` , the `Archive/` folders, stale lower-case `.f` copies, `Main/*.com` header copies, ...) were deleted on 2026-10-23. They remain reachable through the tag `pyaces-legacy-20261023` (Appendix A).

3. Build environment

3.1 Required tools

Item	Requirement
Fortran	Intel `ifx` or `ifort`, from `$PYACES_FC` (set by `../build.sh` from `./configure --fortran`; default ifx when present, else ifort).
C	`$PYACES_CC` (set by `../build.sh` from `./configure`'s C compiler; default icx when present, else gcc), to compile the f2py C wrapper and `fortranobject.c`. They need no MPI.
Python	3.12 (verified) with numpy (including f2py). The extension suffix is taken from the building interpreter (`EXT_SUFFIX`), so the module imports only under the Python version that built it.
BLAS/LAPACK	Intel MKL; `MKLROOT` must be set. Linked as ILP64, sequential.
binutils	`ar`, `nm`, `objcopy`
Environment modules	The `module` command. `pyaces/build.sh` runs `module purge` and loads its own set.
Prerequisite	aces2 must be built first, so that `aces2/lib/lib*.a` exists.

3.2 configure and the toolchain

The top-level `./configure` finds a toolchain and writes `build_config.sh`: how to put it on PATH (`ACES_PROGRAM_ENV_MODE` = `modules`, `setvars` or `path`) and the Intel Fortran and C compilers. `pyaces/build.sh` takes the first of these that applies:

1. `PYACES_MODULES`, when set: `module purge; module load` that set (`aces_load_modules` in `cmake/env.sh`), or nothing for `""`. The CMake superbuild sets it, and so does the top-level `build.sh`, to the recorded module set (empty in setvars/path mode, where it has already loaded the toolchain).
2. `ACES_PROGRAM_MODULES` from the environment, when non-empty.
3. Run on its own: `../build_config.sh`'s own loader (`aces_program_load_env`) and its `ACES_PROGRAM_FC` / `ACES_PROGRAM_CC`.
4. No `build_config.sh`: the HiPerGator set (`intel/2025.1.0 openmpi/5.0.7 python/3.12`).

The log's `compilers:` line shows which Fortran and C compilers were used.

```
# ccsd server (modules mode)
./configure --modules="intel/oneapi_2023.2.0 python/3.12.3" --np=4
# local workstation (setvars mode)
./configure --setvars=/opt/intel/oneapi/setvars.sh --fortran=ifort
```

> **Caution**: include a Python module in the set. If `module purge` unloads the Python that has numpy, the f2py step fails.

3.3 Locale

`pyaces/build.sh` forces `export LC_ALL=C` (line 19). The globs and `sort` calls in the script set archive-member order and `-l` order, and that order decides which copy of a duplicated routine gets linked. For

the same reason the aces2 makefile wraps its source lists in `$(sort …)`: GNU Make 4.2 does not sort wildcard results, and 4.3 sorts them by locale.

3.4 Build commands

```
./build.sh aces2 pyaces          # top level: aces2, then pyaces
./build.sh pyaces                # only pyaces, when aces2 is already built
bash pyaces/build.sh             # run directly: uses build_config.sh as ./build.sh does
```

The log is `build_logs/unified_build_pyaces.log`. On the shared local workstation, stay inside the allocated cores: `taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./build.sh pyaces`. On the ccsd server, build only through PBS (`build_logs/build_ccsd.pbs`).

> **After changing compiler flags**: make does not track flag changes. After changing aces2 flags, remove its build output with `git clean -fdXq aces2` and rebuild. pyaces wipes and recreates `gen/` and `build/` on every build.

4. Build pipeline (`build.sh`)

4.0 Setup (lines 1–36)

- Pick the Fortran compiler: `$PYACES_FC`, else ifx, else ifort. Pick the C compiler: `$PYACES_CC`, else icx, else gcc.
- Load the toolchain as in section 3.2 (`PYACES_MODULES`, else `ACES_PROGRAM_MODULES`, else `build_config.sh`'s loader, else the HiPerGator set). A module set goes through `aces_load_modules` from `cmake/env.sh`: `module purge; module load`, nothing for an empty set, a warning and the current environment on a host without a `module` command.
- Exit if `MKLROOT` is empty. `MKL=$MKLROOT/lib/intel64` (or `$MKLROOT/lib`).
- `export LC_ALL=C`.
- Ask the running python3 for `NUMPY_INC`, `PY_INC` and `F2PY_SRC` (numpy's `f2py/src`).
- `cd Main; rm -rf gen build; mkdir -p build`.

4.1 Source lists

`PYF_SOURCES` lists the sources whose routines make up the Python interface. f2py scans them and wraps every routine it finds; there is no `.pyf`. Passing every source would also wrap internal helpers, whose all-caps F77 names trip over case mangling. Hence the hand-picked list.

`SOURCES` (lines 52–62) is every Fortran file to compile: `a2_pyf/*.F` except `rungrads`, `run1props` and `run`, plus `Main/init.F`, `Main/aces2_main.F` and `aux_pyaces/*.F`. So `a2_get*/a2_put*`, `getlstpy_dbg`, `get1epy` and `returnpy` are linked but not exposed to Python.

4.2 Steps

Step	Command / action	Output
1. Generate wrappers	`python3 -m numpy.f2py $PYF_SOURCES -m aces2py --build-dir gen` (generate only, no `-c`). Then a `sed` lower-cases the first argument of every `F_FUNC(Name,…)`: ifx exports lower-case symbols such as `init_`, but f2py uses the source spelling `Init`.	`gen/aces2pymodule.c` `gen/aces2py-f2pywrappers.f` `f2py_gen.log`
2. Compile C	`$PYACES_CC -DNDEBUG -fPIC -O2 -I. -I$NUMPY_INC -I$PY_INC -I$F2PY_SRC` on the module C file and `fortranobject.c`.	`build/aces2pymodule.o`, `build/fortranobject.o`
3. Compile Fortran	Compile `$SOURCES` and the f2py wrapper with the `FFLAGS` below. Then `objcopy --redefine-sym props_=pyaces_vprops_props_ build/runee.o`, so Runee calls the renamed vprops PROPS.	`build/*.o`
4. Link	Build LIBDIRS/LIBNAMES, build the patched archives for colliding modules (chapter 5), then `ifx -shared -nofor-main`.	`build/aces2py.cpython-312-x86_64-linux-gnu.so`
5. Import test	`python3 -c "import aces2py; print('IMPORT OK')"`, then `collision_audit.py --calls` (section 6.1). Either failure fails the build (`set -e`).	none
6. Link audit	`python3 tools/collision_audit.py || true`. Advisory only; it never fails the build.	`@COLLISION_AUDIT:` lines in the log

Fortran flags

```
FFLAGS="-diag-disable=10448 -i8 -fPIC -fp-model strict -O1 -assume minus0 \
        -D__fortran -D__fortran77 -D_INTEL -D_RECL_IS_WORDS_ -DF_64BIT \
        -I$ACES/include -I$ACES/acescore/include -I. -FI"
```

- `-i8`: 8-byte default INTEGER, matching aces2 and MKL ILP64.
- `-DF_64BIT`: set by aces2's makefiles together with `-i8`. Here it only affects `a2_getcrec`/`a2_putcrec`, which count a character record in 8-byte words. Without it (before 2026-10-30) they counted 4-byte words, so `Cgetrecpy` asked for twice the length of a record written by aces2 and ended the process.
- `-FI`: fixed-form source. `-D__fortran77` selects the F77 branch of aces2's headers.
- No `-qopenmp`: the pyaces sources have no OpenMP directives, aces2 is compiled without it and MKL is sequential. It was dropped on 2026-10-30 (T-09) with identical suite results; it only made every local variable automatic.
- Headers come from aces2's `include` and `acescore/include`. `Main/init.F` and `Main/aces2_main.F` include them the same way as every other pyaces source.

Link command

```
ifx -shared -nofor-main -o build/aces2py${EXT_SUFFIX} build/*.o \
    -L$PATCHED_LIBDIR $LIBDIRS -Wl,--start-group $LIBNAMES -Wl,--end-group \
    -L$MKL -Wl,--start-group -lmkl_intel_ilp64 -lmkl_lapack95_ilp64 \
           -lmkl_sequential -lmkl_core -Wl,--end-group -lpthread
```

- `LIBDIRS`/`LIBNAMES` are regenerated every build from `find $ACES -maxdepth 2 -iname "*.a"` (sorted, de-duplicated). All aces2 archives sit in one group, so circular dependencies resolve.
- `-L$PATCHED_LIBDIR` comes first, so a patched copy wins over the original archive of the same name.
- `-nofor-main` stops ifx from requiring its own main program.

5. Linking into one .so: symbol collisions and their fixes

The classic ACES II build links each module into its own executable. Routines that **share a name but not an implementation**, such as vcc's `FINISH` and lambda's `FINISH`, were therefore harmless. pyaces puts all of them into one .so, so the linker picks one copy and hands it to every caller, without a warning. The result is a segfault, `@GETLST: List (x,y) does not exist`, or a **silently wrong energy**. The pyaces link step has four safeguards against this.

5.1 Localizing MAIN__

Twelve modules (`hbar joda lambda pccd props vcc vee vmol vmol2ja vscf vtran intprc`, `MAIN_COLLISION_MODULES`) contain a Fortran PROGRAM, so each defines `MAIN__`. The standalone executables need it, but in one .so it means "multiple definition of MAIN__". The build extracts each of these archives with `ar x` into a temporary directory, applies `objcopy --localize-symbol=MAIN__` to the objects that define it, and repacks them as `build/patched_libs/lib<mod>.a`. aces2's own `.a` files are not modified.

5.2 `tools/localize.txt`: renaming the losing copy

Each `rename <module> <symbol>` line in `tools/localize.txt` (60 today, each with its reason) becomes a `module:symbol` pair in build.sh's `EXTRA_LOCALIZE_SYMBOLS`, and each `extract <module>` line an entry of `EXTRA_LOCALIZE_MODULES`. For each pair the build applies `objcopy --redefine-sym sym=pyaces_<mod>_sym` to **every object** in that module's archive. The definition and the module's own calls are renamed together, so the losing module keeps using its own implementation under the new name, and the original name is left to the winner.

> **Rule**: a rename only takes effect if its module is extracted, i.e. in `MAIN_COLLISION_MODULES` or on an `extract` line. Before patching, the build checks every pair: if the module is not extracted, or its archive defines no `T` symbol of that name, the build stops with `tools/localize.txt has renames that cannot be applied`. Keep extract lines that have no rename of their own (libr2): repacking an archive reorders its members, which can change which duplicate wins.

The build used `--localize-symbol` at first, but that sent the losing module's own calls to the winner (lambda's `vlamcc` ended up calling vcc's FINISH), so on 2026-08-11 it switched to `--redefine-sym`.

Main collisions resolved so far

Symbol	Renamed (losing) copy	Symptom / note
`FINISH`	lambda, molcas	Runcc reported the iteration-0 TOTENERG instead of the converged one
`GSCHMIDT`	joda, libr, vcceh, vea	SIGSEGV from an argument-count mismatch
`MODF`	lambda, fsip, vcceh, hcmult, vea, lcct	Failures on EOM paths
`EXPDEN`, `PROPS` and vprops helpers	props, vcceh / vprops	Dipole calculation in Runee

`HBARXC`, `DT2INT2`, `DRAOLAD`, `T2TOAO`, `GETAOINF`, `Z2TOMO`	vcceh, lcct, hcmult, lambda, vee, dens	GETLST "list does not exist"; Z2TOMO gave an error of about 1e-5
`W45T3DRV`	lambda	lambda's 8-argument copy won over trp's 4-argument one
`CMPENG`	rpa	Singles term dropped: a silent 2.49e-05 error
`XGEMM`	tools	Illegal argument passed to DGEMM
`CHECK_T2`, `INITIN/INITSN/DIISLST/DODIIS0`	ringcc / lambda	vcc used other modules' DIIS and initialisation routines
`ABCD` (COMMON shadow)	intgrt	Every KS case faulted on instruction fetch (section 5.4)
`getdt1`, `fndmaxs`, `t2l2rng`, `r1l2y2c`	vcc, vee, lcct, lcct	The winner depended on link order (make 4.2 vs 4.3); pinned in eab3c4e6

The cause and diagnosis of each entry are recorded next to its entry in `tools/localize.txt`.

5.3 `NEWLY_LINKED_MODULES`: bulk handling of re-enabled modules

`a2mix aux rpa scrnc` were `.skip` directories until they were re-enabled, so the classic build never linked them next to the other modules. The build collects the `T` symbols of every `aces2/lib/lib*.a` and automatically adds to `EXTRA_LOCALIZE_SYMBOLS` every symbol (except `MAIN__`) that one of these four modules shares with another module. The log shows `shadowed by <mod>, renaming: …`.

5.4 COMMON blocks that shadow subroutines

A Fortran `COMMON /abcd/` emits a tentative symbol `abcd_` (`C` in `nm`). A subroutine `ABCD` is also `abcd_`. If the COMMON satisfies the reference before the linker extracts the object holding the subroutine, `call abcd` jumps into zero-filled BSS. The symptom is a SIGSEGV on instruction fetch, recognisable in gdb by `si_addr == $pc`.

- **rename-definer**: if only the defining module calls the routine, add `rename <definer> <sym>` to `tools/localize.txt` (e.g. `rename intgrt abcd_`).
- **rename-commons**: if other modules call it too, renaming would orphan them, so the COMMON block has to move instead.

Nine latent cases (mopac `deriv/grid/local/polar`, nddo `guess/local`, aces2 and ccd_light `energy`, pdens `one/tab`, vdint `sym2`) were renamed ahead of time.

5.5 Keeping patched_libs clean

`build/patched_libs` comes before aces2/lib on the `-L` path, so a stale file there shadows the real archive. A `libintgrt.a` left over from an experiment once caused `undefined symbol: intgrt_run_`. The directory is now removed and rebuilt on every build.

5.6 Finding out which copy won

```
# First choice: ask the linker (add to the link command)
-Wl,-trace-symbol=getdt1_
# Fallback: match the size in the .so against the archives
nm -S Main/build/aces2py*.so | grep ' getdt1_$'
nm -S ../aces2/lib/libvcc.a ../aces2/lib/libvee.a | grep ' getdt1_$'
# COMMON shadow check: B or D instead of T means data, not code
nm Main/build/aces2py*.so | grep -E ' (abcd_|hbar_)$'
# Which symbol does a pyaces object actually call?
nm Main/build/runhbar.o | grep -i hbar
```

6. The link-resolution audit (`collision_audit.py`)

It runs automatically as build step 6 and can also be run by hand. It needs an already built `Main/build/aces2py*.so`.

```
python3 tools/collision_audit.py            # report against the baselines
python3 tools/collision_audit.py --update   # overwrite both baseline files with the curren
t findings
python3 tools/collision_audit.py --calls    # fail if a pyaces call binds to data (run by b
uild step 5)
```

6.1 What it checks

(1) Resolution differences. For every symbol defined by more than one module:

1. Reconstruct the classic link order from `ACES_LINK_LISTS` in `aces2/Makefiles/GNUmakefile`.
2. Identify the copy actually in the .so by its `nm -S` size (skipped if sizes cannot tell the copies apart).
3. For each calling module, compare it with the copy the classic build would have given (its own copy, otherwise the first library in its link list that defines the symbol).
4. If the two differ, and their comment-stripped sources differ too, report `symbol winner classic caller`.

Pairs already renamed by `build.sh` are excluded; the audit reads the rename lines of `tools/localize.txt` and `NEWLY_LINKED_MODULES` from build.sh.

(2) COMMON shadows. It reports symbols that are `T` in one archive and `C` in another and B/b/D/d in the .so, with a verdict of `rename-definer` or `rename-commons`.

(3) Calls bound to data (`--calls`). Every `U` symbol in pyaces's own Fortran objects (`Main/build/*.o` except the two C objects) is a call, because Fortran references COMMON blocks as `C`. If the .so defines that name as data (B/b/D/d), the call would jump into a COMMON block. This catches the case (2) cannot, where no subroutine of that name exists anywhere (the `hbar_` defect). It is not advisory: build step 5 runs it and any hit fails the build.

6.2 Reading the output

```
@COLLISION_AUDIT: 0 subroutines shadowed by a same-named COMMON block ...
@COLLISION_AUDIT: 296 resolution differences (296 in the baseline)
  resolved since the baseline: ...             # in the baseline, now gone
@COLLISION_AUDIT: NEW resolution differences ...
    getdt1_   .so gives vcc, lcct expects lcct   # new: needs action
```

6.3 When NEW entries appear

1. Compare the two implementations (argument count, list numbers, what they compute). If they compute the same thing, the entry can be accepted into the baseline.
2. If they differ, choose the copy to rename and add `rename <loser> <symbol>` to `tools/localize.txt`, with an `extract <loser>` line if the module is not extracted yet.
3. Rebuild pyaces and confirm the NEW entry is gone and no other symbol flipped. Adding a module to the processed set can change how its other symbols resolve.

4. Update the baseline with `--update` only after the suite has passed.

> **The baseline is not a to-do list**: its 296 entries are known, accepted differences. Renaming 42 "safe-looking" entries in one pass once dropped the suite from 45 to 31 PASS. Also, `ACES_LINK_LISTS` only covers modules that build executables, so callers inside library modules (libr, libr2, tools, acescore, ...) are invisible to the audit. Calls from pyaces's own objects into COMMON blocks are covered separately by `--calls`.

7. The `aces2py` API

7.1 Signatures come from the Fortran sources

f2py builds the module from the sources in `PYF_SOURCES` alone. (Until 2026-10-23 a `Main/aces2.pyf` was passed as well; f2py then built the module twice and the source scan overwrote the `.pyf` version, so the `.pyf` never took effect. Dropping it left `aces2pymodule.c` byte-for-byte the same.) As a result:

- Python names follow the capitalisation in the source: `Init`, `Dgetrecpy`, `IGetrecpy` (capital G).
- Argument types and intents come from the source. To change an intent, put a `Cf2py intent(...)` line in the `.F` file. Every `Run*` takes an `Iuhf` argument.
- No COMMON blocks (`/flags/`, `/jobarc/`, `/lists/`, ...) are exposed; read state through JOBARC records instead.
- Check exact signatures with `print(aces2py.Runcc.__doc__)` or the docstrings in `gen/aces2pymodule.c`.

7.2 Array arguments

Every function is a Fortran subroutine. Apart from the two functions in 7.4 that return a value, results are **written into numpy arrays the caller passes in**. f2py wraps those arrays as "input", so if the dtype or memory layout is wrong it silently passes a **copy**, and whatever Fortran writes is lost with the copy.

> **Always** pre-allocate C-contiguous 1-D arrays: `np.zeros(n, dtype=np.float64)` for reals, `np.zeros(n, dtype=np.int64)` for integers. Record names are 8 characters, padded with blanks (`"IFLAGS  "`).

7.3 Calculation drivers

`Iuhf`: 0 = RHF, 1 = UHF/ROHF (`Integer*8` on the Fortran side, `long long` in C). No driver calls `aces_fin`, so the session stays open afterwards and records can be read.

Function	Source	What it runs
`Init()`	`Main/init.F`	`aces2_main`: `Joda_` (parse ZMAT, create JOBARC), `Aces_ja_fin`, `aces_init` (allocate core, set up flags and lists, reopen JOBARC). **Call it first, before every driver except Runee.**
`Runints(Iuhf)`	`runints.F`	`Ints_` (vmol AO integrals) and `V2ja`
`Runscf(Iuhf)`	`runscf.F`	Integrals first if there is no IIII file, then `Vscf`
`Runcc(Iuhf)`	`runcc.F`	SCF first if it has not run (no `SCFEVCA0` record), then `aces_init_chemsys`, `Vtran`, `Init_legacies`, `Intproc`, `Vcc`. Covers the MBPT/CC family.
`Runhbar(Iuhf)`, `Buildhbar()`	`runhbar.F`, `buildhbar.F`	The Runcc steps, then `Hbar_`. If SCF is done (`SCFEVCA0`) it starts at Vtran; if CC is done (`CC_DONE`) it runs only Hbar. Buildhbar reads iuhf from `IFLAGS(11)`. **Do not call it after Runcc**: it takes the SCF-done branch and runs Vtran a second time.
`Runprops(Iuhf)`	`runprops.F`	`Props_`

`Runpccd(Iuhf)`	`runpccd.F`	Single-point pCCD after `Init()`, as xrunpccd's HF/KS branch: Ints, V2ja, Vscf, Vtran, Intproc, Vcc. Orbital-optimized pCCD (`PROGRAM=PCCD`) is not reachable.
`Runee(Iuhf)`	`runee.F`	Self-contained EOM-CC: Joda, aces_init, Ints, V2ja, vprops Props (renamed copy), Vscf, Vtran, Intproc, Vcc, Vlambda, Vee. **Do not call `Init()` before it.**
`Rundirmp2(Iuhf)`	`rundirmp2.F`	SCF if needed, then `Dirmp2_run(0)`. Iuhf is not forwarded.
`Runintgrt(Iuhf)`	`runintgrt.F`	SCF if needed, then `Intgrt_run` (KS/DFT numerical integration, XC energy)
`Acesjafin()` / `Acesjainit()`	`acesjafin.F` / `acesjainit.F`	Close (flush) / reopen the JOBARC session. Reopening an already open session is a hard `aces_exit`.

7.4 JOBARC records

Signature	Description
`Dgetrecpy(Name, Dest, [Length])`	Read a float64 record into `Dest`, e.g. `"TOTENERG"`, `"SCFENEG "`
`IGetrecpy(Name, IDest, [Length])`	Read an int64 record, e.g. `"IFLAGS "` (100), `"NBASTOT "`
`Dputrecpy(Name, Source, [Length])`	Write a float64 record
`Iputrecpy(Name, Isource, [Length])`	Write an int64 record
`Cputrecpy(Name, Length, Source)`	Write the first `Length` bytes of `Source` as a character record (`Source` is assumed-length; it was `Character*8` before 2026-10-30 and stored only 8 characters).
`Dest = Cgetrecpy(Name, Length)`	Read `Length` bytes of a character record and return them as `bytes` (length 1024, blank-padded). A length above 1024 is an `aces_exit`. The record length is counted in 8-byte words, as aces2's `getcrec` counts it (`-DF_64BIT`, section 4.2), so character records written by aces2 programs read back correctly.
`Length = Getreclenpy(Name)`	Return the record length in integer words, or `-1` if the record does not exist.

Lengths are in integer words; with `-i8`, one integer = one real = 8 bytes. If `Length` is omitted, the array length is used.

7.5 MO lists (MOIO)

Signature	Description

`Getlstpy(Column_Index, No_columns, Irrep, Left_index, Right_index, Destn, Ndim)`	Calls `A2_getlst(Destn, Column_Index, No_columns, 1, Left_index, Right_index)`, i.e. ACES II's `getlst(Z, first, n, 1, irrep, list)`: **`Left_index` is the irrep of the columns and `Right_index` the list number** (`MOIO(irrep, list)`). `Irrep` is not used; it stays so that existing callers keep working. If the list does not exist, the column range is outside it or `Ndim` is shorter than the columns, it prints `@Getlstpy: …` and leaves `Destn` as it was.
`Putlstpy(Source, Column_Index, No_columns, Irrep, Left_index, Right_index, [Ndim])`	The write counterpart of Getlstpy, with the same checks (`Ndim` is the length of `Source`).
`Nrows, Ncols = Getlstdimpy(Left_index, Right_index)`	The shape of `MOIO(Left_index, Right_index)`: rows per column and number of columns; both 0 if the list does not exist.
`Getallpy(Destn, Irrep, Right_index, [Ndim])`	Read all irrep blocks of list `Right_index` with total symmetry `Irrep`
`Putallpy(Source, Irrep, Right_index, [Ndim])`	The write counterpart

A table of list numbers is in `pyaces.moio` (e.g. T2(Ij,Ab) = 46, T1(AI) = 901, W<AB||IJ> = 14). Use `list_numbers(label)`, `list_labels(number)` or `distribution_numbers(label)`; some labels and numbers repeat, so each returns a list. The entries were transcribed by hand and have not been checked one by one against the ACES II sources.

7.6 AO integrals

Signature	Description
`Get1ehpy(Name, Dest, Ndim)`	Read `"overlap"`, `"oneh"` or `"kinetic"` from the IIII file and expand it into a full `Ndim×Ndim` matrix (`Ndim = NBASTOT`). Without symmetry the basis is the AO basis. With symmetry it is the SO basis with the functions in irrep order (`NUMBASIR`), and the matrix is block diagonal; the SCF eigenvectors `SCFEVCA0` are in the same basis, so $C^{T}SC = 1$.
`Get2ehpy(Name, Dest, Ndim)`	`"2elints"`: expand all two-electron integrals from IIII/IJIJ/IIJJ/IJKL into an array of size $Ndim^4$. Mind the memory.

7.7 The `pyaces` Python package

`python/` holds a small package on top of `aces2py`. It does not replace the raw module (still available as `pyaces.ext()`, names unchanged); it adds lower-case functions, return values instead of output arrays, and a way to run a calculation without risking the caller's process.

```
pip install -e pyaces/python          # or: export PYTHONPATH=.../pyaces/python
# aces2py is found in pyaces/Main/build, or in $PYACES_EXT_DIR
```

Function	What it does

`run(method, zmat, genbas, iuhf=None, record="TOTENERG", workdir=None, timeout=None, keep=False)`	Runs one calculation in a child process (`pyaces_run_case.py`, including its FNO handling) and returns `Result(value, record, workdir, log)`. `method` is `scf`, `cc`, `ee`, `pccd`, `dirmp2`, `ks`, `hbar` or `props`; `zmat` is text or a path; `iuhf` is guessed from REF/REFERENCE and QRHF. Any failure, including an ACES II `aces_exit` or a crash, raises `CalculationError` (with `status`, `returncode`, `workdir`, `log_tail`) and the caller keeps running.
`init()`, `run_scf(iuhf)`, `run_cc`, `run_hbar`, `run_ee`, `run_pccd`, `run_dirmp2`, `run_intgrt`, `run_props`, `run_ints`, `close_jobarc()`, `open_jobarc()`	In-process drivers: the same calls as section 7.3, same rules (one calculation per process, an ACES II error ends the process).
`record_length(name)`, `get_record(name, dtype=float64)`, `put_record(name, values)`, `get_string_record(name, nbytes)`, `put_string_record(name, data)`	JOBARC records with names padded for you. `get_record` sizes the array from the record and raises `KeyError` for a missing record instead of letting aces2py end the process.
`list_shape(lst, irrep)`, `get_list(lst, irrep, first_column, n_columns, size=None)`, `put_list`, `get_all_irreps`, `put_all_irreps`	MO lists (7.5). `list_shape` returns `(rows, columns)` or `None`; `get_list` sizes its array from it by default. `get_list` and `put_list` raise `KeyError` for a missing list and `ValueError` for a bad column range or a short array, where aces2py would only print a message.
`get_1e_ints(name)`	`Get1ehpy` as an `(nbas, nbas)` array; `name` is `overlap`, `oneh` or `kinetic` (7.6).
`moio.list_numbers(label)`, `moio.list_labels(n)`, `moio.distribution_numbers(label)`	MO-list and distribution number tables.

```
import pyaces
r = pyaces.run("cc", "ZMAT", "/path/to/GENBAS")
print(r.value)                       # TOTENERG
try:
    pyaces.run("cc", bad_zmat, "/path/to/GENBAS")
except pyaces.CalculationError as e:
    print(e.status, e.log_tail)      # the caller is still alive
```

8. Running calculations

8.1 Runtime environment

- **Working directory**: must contain `ZMAT` and `GENBAS`. ACES writes JOBARC, JAINDX, IIII/IJIJ/IIJJ/IJKL, MOINTS, MOABCD, MOL, NEWMOS and others into the same directory. Memory size comes from the ZMAT keywords.
- **Libraries**: the Intel runtime and MKL are needed. Load the same module set as for the build, or set `LD_LIBRARY_PATH`. Otherwise the import fails with `libmkl_intel_ilp64.so.2: cannot open shared object file`.
- **Import path**: put `pyaces/Main/build` on `sys.path` or `PYTHONPATH`.

8.2 Basic usage

```
import os, sys, numpy as np
os.chdir("/path/to/workdir")                # contains ZMAT and GENBAS
sys.path.insert(0, "/path/to/ACES_PROGRAM/pyaces/Main/build")
import aces2py

aces2py.Init()                              # skip for Runee
aces2py.Runcc(0)                            # 0 = RHF, 1 = UHF/ROHF

e = np.zeros(1, dtype=np.float64)
aces2py.Dgetrecpy("TOTENERG", e)
print("E =", e[0])
print("TOTENERG length:", aces2py.Getreclenpy("TOTENERG"))
```

8.3 Reading and modifying records

```
iflags = np.zeros(100, dtype=np.int64)
aces2py.IGetrecpy("IFLAGS  ", iflags)
iflags[1] = 1                               # IFLAGS(2) = CALC
aces2py.Iputrecpy("IFLAGS  ", iflags)
aces2py.Acesjafin()                         # flush before an external executable reads JO
BARC
# ... run ../aces2/bin/xvtran etc. as subprocesses ...
aces2py.Acesjainit()
```

8.4 Rules

- **One calculation per process.** Do not run several ZMATs in a row in one process. The test runner starts a new process for each case.
- **Do not run the same module twice.** Calling Vtran/Vcc twice in one process has converged to wrong values. This includes calling Runhbar after Runcc.
- **Runee calls Joda itself.** Do not call `Init()` before Runee. Every other entry point, Runpccd included, needs `Init()` first.
- **Out of scope**: geometry optimisation, frequencies, analytic derivatives (gradients and Hessians), Brueckner, `HFSTABILITY=FOLLOW` and CCSD(TQ) cannot run under pyaces yet (`rungrads.F` is excluded

from the build).

8.5 FNO special case (`pyaces_run_case.py`)

When ZMAT contains `FNO_KEEP=`, the classic driver runs a preliminary Vtran/Intproc/Vcc/Fno pass to choose which virtual orbitals to freeze. Doing that in-process would call Vtran/Vcc twice, so the runner does this instead:

1. Run `Runscf`.
2. Temporarily change `IFLAGS` 2, 93, 83 and 100, and move `NUMDROPA` to `FNOFREEZ`.
3. Call `Acesjafin()`, then run the standalone `../aces2/bin/{xvtran,xintprc,xvcc,xfno}` as subprocesses (logs: `fno_prelim_<exe>.log`).
4. Restore the flags; regenerate IIII with `xvmol` / `xvmol2ja` if it was consumed.
5. Call `Acesjainit()`, then `Runcc`.

8.6 `pyaces_run_case.py` output protocol

```
python3 -u pyaces_run_case.py <workdir> <entrypoint> <iuhf> <record>
# progress: PYACES_CASE: ...
# success:  PYACES_CASE_VALUE: <repr>   /   PYACES_CASE_RESULT: SUCCESS <%.10f>
# failure (exit 1): IMPORT_FAILED, <ENTRY>_FAILED, RECORD_ZERO_OR_MISSING, DGETRECPY_FAILED
```

A record value of exactly 0.0 is treated as missing.

9. The test suite

9.1 Running it

```
./run_tests.sh pyaces                                    # top level (log: build_logs/unifie
d_test_pyaces.log)
cd pyaces && python3 pyaces_suite_runner.py              # directly
cd pyaces && python3 pyaces_suite_runner.py 010a 117b    # selected cases only
```

- Case input: `aces2/tests/fast/cases/<name>/ZMAT`; basis: `basis/GENBAS`. The suite checks energies; the API tests in `pyaces/tests` (section 9.5) check the rest.
- Working directory: `$PYACES_SUITE_RUN_DIR/<name>`, by default `pyaces/suite_run` (untracked). On HiPerGator, `run_tests.sh` points it at the `/blue/...` scratch filesystem.
- Each case gets a fresh directory with ZMAT copied in and GENBAS symlinked, then `pyaces_run_case.py` runs with a 300-second limit. Log: `<dir>/run.log`; summary: `$RUN_DIR/summary.tsv`.
- Local workstation: `OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./run_tests.sh pyaces`. ccsd server: `qsub build_logs/test_ccsd.pbs`.

9.2 Grading

Status	Condition
PASS	Exit code 0 and $\lvert\text{computed} - \text{reference}\rvert \le 1\times10^{-6}\ E_h$ (`TOLERANCE`, no per-case overrides)
MISMATCH	Exit code 0, but the difference exceeds the tolerance
NO_RESULT	Exit code 0, but no `PYACES_CASE_RESULT: SUCCESS` line
TIMEOUT	More than 300 seconds
FAIL_CLEAN	Positive exit code with a recognised message: an `@ROUTINE:` banner, "Assertion failed", "did not converge", "forrtl: severe", ...
CRASH	Killed by a signal (negative code), or a positive exit with no recognised message

Latest result: 60/60 PASS (2026-10-30, local workstation and ccsd), largest difference 4.95×10^{-8} (case 078).

9.3 `suite_classification.tsv`

Columns: `name`, `category`, `reason`, `entrypoint`, `iuhf`, `record` (the JOBARC record compared, `TOTENERG` or `SCFENEG`), `ref_value` (Fortran E/D format). The runner only runs rows whose category is SCF, CC, EOM, PCCD or KS.

Category	Count	Entry point / iuhf
CC	39	Runcc (iuhf 0: 16, 1: 20), Rundirmp2 (3)
EOM	8	Runee (0: 6, 1: 2)
SCF	9	Runscf (0: 6, 1: 3)
KS	3	Runintgrt (0)
PCCD	1	Runpccd (0: 1)

EXCLUDED	67	Geometry optimisation, VIB, derivatives, MRCC, Brueckner, HFSTABILITY=FOLLOW, CCSD(TQ), PROGRAM=PCCD, ...
SCRIPT	31	Cases driven by scripts rather than a ZMAT

9.4 `classify_suite.py` is the source of the TSV

`classify_suite.py` builds the TSV from `aces2/tests/fast/test_results` and the ZMATs. Every routing rule lives in `classify()`, including the ones that were once hand edits: `SCF_TYPE=KS` → KS/Runintgrt, `FOCK=AO` MBPT(2) → Rundirmp2, and the exclusions for CCSD(TQ), `BRUECKNER` and `HFSTABILITY=FOLLOW`. `PROGRAM=PCCD` is excluded too: `aces2/aces2/main.F` hands such a job to `xrunpccd`, which runs xjoda again and the OO-pCCD loop, and `Runpccd` does not. **Never edit the TSV by hand**: change `classify()` and regenerate.

```
python3 classify_suite.py           # rewrite suite_classification.tsv
python3 classify_suite.py --check   # exit 1 and show the rows if the TSV differs from the
rules
```

> **Roll-up**: `run_tests.sh` reads every `STATUS: n` line of the runner's Summary block. PASS counts as PASS, MISMATCH and FAIL_CLEAN as FAIL, and everything else (CRASH, NO_RESULT, TIMEOUT, ...) as OTHER, so TOTAL is always the number of cases run.

9.5 API tests

The suite only compares energies. `pyaces/tests` tests the in-process API itself, each test in its own process and directory (an ACES II error ends the process):

```
cd pyaces && python3 tests/run_api_tests.py           # all (log lines and a Summary block)
cd pyaces && python3 tests/run_api_tests.py lists     # one test
# work directories: $PYACES_API_RUN_DIR, default pyaces/suite_run/api; extension: $PYACES_E
XT_DIR as usual
```

Test	Checks
`records`	`Getreclenpy` returns an int, `record_length`, a missing record (`None`, `KeyError`), float64/int64/character round trips, the 8-byte-word length of a character record, records after `close_jobarc` / `open_jobarc`
`scf_rhf`, `scf_uhf`	`run_scf(0)` / `run_scf(1)` energies against xaces2; reading vscf's character record `EVCSYMAF`; UHF writes beta orbitals
`hbar_from_scratch`, `hbar_after_scf`	Runhbar's "nothing done" and "SCF done" branches: the CCSD energy and the `HBAR` record hbar writes when it finishes. The "CC done" branch needs a `CC_DONE` record without `SCFEVCA0`; no ACES II program writes `CC_DONE`, so it cannot be reached.
`lists`	After `run_cc`: list 46 read irrep by irrep and column by column equals `get_all_irreps`; `list_shape`; `KeyError` / `ValueError` for bad requests; `Getlstpy` leaves a short array alone; `put_list` round trip
`one_electron_c2v`, `one_electron_c1`	`get_1e_ints`: symmetric, block diagonal by irrep (C_{2v} STO-3G H_2O has an empty A_2), $C^TSC = 1$, and E(SCF) = $\Sigma_{occ}(h_{ii} + \varepsilon_i) + E_{nuc}$ rebuilt from `oneh` against xaces2's E(SCF)

The reference energies come from the classic `xaces2` on the same inputs, not from pyaces. `run_tests.sh pyaces` runs them after the suite (log `build_logs/unified_test_pyaces_api.log`) and adds a `py-api` row to the roll-up. On a build with `f97a9759` reverted the `hbar_after_scf` test crashes (SIGSEGV in the `/HBAR/` common) and every test that uses `record_length` fails (`Getreclenpy` returned nothing); the build's own `--calls` check also stops at the two `hbar_` calls. The UHF test passes there: the 4-byte `Iuhf` happened to arrive as 1.

10. Extending pyaces: new entry points and new tests

10.1 Exposing a new Fortran routine to Python

1. **Make the aces2 side callable.** If the target is a PROGRAM, split its body into a subroutine (the "wrapper-split" pattern used for `dirmp2_run`, `intgrt_run`, `joda_`, `hbar_`, `props_`). Rebuild aces2 first.
2. **Write `a2_pyf/<name>.F`.**
   - Fixed form, with every `#include` starting in column 1 (oneAPI 2023.2 ifort rejects indented includes).
   - Declare every dummy argument **explicitly** as `Integer*8` or `Double Precision`. A plain `Integer` makes f2py pass a 4-byte C `int` while `-i8` Fortran reads 8 bytes.
   - Mark outputs with `Cf2py intent(out) <arg>`; the source is the only place signatures are defined (7.1).
   - The capitalisation of the subroutine name becomes the Python name.
   - Use `Icore(i0), Icrsiz` from `icore.com`/`istart.com` for core memory (see `runcc.F`). Do not call `aces_fin`.
   - Call aces2 routines by their real symbol: if aces2 defines `SUBROUTINE HBAR_`, the call must be `Call Hbar_`; `Call Hbar` would bind to a COMMON block (chapter 11).
3. **Add a lower-case wrapper** to `python/pyaces/drivers.py` (and to `runner.ENTRY_POINTS` if it is a calculation entry point).
4. **Add the file to `PYF_SOURCES` in `build.sh`.** The `a2_pyf/*.F` glob compiles it automatically unless it is on the exclusion list.
5. **Handle collisions.**
   - If a newly pulled-in module has a PROGRAM, add it to `MAIN_COLLISION_MODULES`.
   - For a same-named routine, add `rename <loser> <sym_>` (and `extract <loser>` if needed) to `tools/localize.txt`.
   - If a pyaces object calls a renamed routine directly, add an `objcopy --redefine-sym` after step 3 (as done for `runee.o`).
   - If you re-enable a `.skip` module, add it to `NEWLY_LINKED_MODULES`.
6. **Build and check**: "IMPORT OK", the new wrapper in the second (source-case) module build in `f2py_gen.log`, and `print(aces2py.<Name>.__doc__)`.
7. **Handle NEW audit entries** as described in section 6.3.
8. **Check that every name you call is code**: `nm Main/build/<name>.o` lists the undefined symbols your object calls, and `nm Main/build/aces2py*.so | grep ' <sym>$'` must show `T`. `B` or `D` means the call is bound to a COMMON block.
9. **Add a test case** and run the whole suite.

10.2 Adding a test case

1. Add the case to the aces2 suite: `aces2/tests/fast/cases/<name>/ZMAT` and a three-line entry in `aces2/tests/fast/test_results` (`name src` / `RECORD 1` / reference value).

2. Run `python3 classify_suite.py` to regenerate `pyaces/suite_classification.tsv`, and check the new row (category, entrypoint, iuhf, record, ref_value). If the case is routed wrongly, change `classify()`, not the TSV. The entry's first line may be `name src` or just `name`.
3. Check it with `PYACES_SUITE_RUN_DIR=... python3 pyaces_suite_runner.py <name>`, then read `<dir>/run.log` and `summary.tsv`.
4. Make sure the full suites (`./run_tests.sh aces2 pyaces`) still pass.

10.3 Checklist after a change

What changed	What to verify
Fortran in a2_pyf / Main	pyaces build, IMPORT OK, suite 60/60, `nm` check of the changed objects
Rename lists in build.sh	Audit (no NEW entries; any "resolved" entries are intended), suite, then `--update`
aces2 sources or flags	`git clean -fdXq aces2`, rebuild aces2 and pyaces, aces2 suite 158/158, pyaces 60/60, audit
Toolchain / another machine	All of the above, plus identical audit results on both machines

11. Known issues (with fix history)

Fixed on 2026-10-23 (commit f97a9759): three defects found while writing this manual.

- **`Call Hbar` in Runhbar/Buildhbar**: the symbol `hbar_` is not a subroutine but lambda's `COMMON /HBAR/` (`B hbar_` in the .so), so the SCF-done and CC-done branches could jump into BSS. Changed to `Call Hbar_` (= `hbar__`). The audit cannot catch this case because there is no `T hbar_` anywhere, which is why step 8 of section 10.1 asks for an `nm` check. Verified with `Init → Runscf → Runhbar` on case 026 (CCSD energy matches the reference to 7×10^{-11}).
- **Width of `Iuhf`**: the `Integer Iuhf` in `run*.F` made f2py pass a 4-byte C `int` while `-i8` Fortran read 8 bytes. Now declared `Integer*8 Iuhf`, so both sides are 64-bit.
- **`Getreclenpy`, `Cgetrecpy`**: their outputs never reached Python. `Cf2py intent(out)` now makes them return the length and the string (section 7.4); the debug prints in Cgetrecpy were removed.

Fixed on 2026-10-23 (TODO stage 1: T-03, T-04, T-07, T-16)

- **Calls bound to data**: `collision_audit.py --calls` now fails the build when a pyaces object calls a name the .so defines as data (section 6.1). It reports both `hbar_` calls when run against the pre-f97a9759 objects.
- **Rename pairs that cannot apply**: the build stops instead of ignoring them (section 5.2), and `objcopy` errors are no longer discarded.
- **Python version**: the extension suffix comes from `EXT_SUFFIX` instead of a hard-coded `cpython-312`.
- **Test roll-up and classifier**: the roll-up counts every status, the former hand edits are rules in `classify()` with a `--check` mode, and the runner's default run directory is inside the repository (sections 9.1–9.4).

No known issues are open. The last ones were fixed on 2026-10-30: `Getlstpy` and `Get1ehpy` (T-05), and the build environment (T-10; the top-level ifort check had already gone with `f444e634`). See section 12.1 and `DONE.md`.

12. Development history and server deployment

12.1 History

Period	Work
2023-11	Original acespy (Ajith Perera), built as per-module shared libraries (`Fxx/`) wrapped with f2py.
2026-08-08 – 08-11	Subtree-merged into ACES_PROGRAM (`ff876bec`). Hard-coded paths removed and the link list generated from the aces2 archives (`37d57822`). Fixing the first collision (FINISH) took the suite from 13 to 30 of 58 PASS. EOM-path collisions fixed (GSCHMIDT, MODF, Z2TOMO, ...); switched from localize to redefine-sym.
2026-09-02 – 09-04	Portable test runners; W45T3DRV, CMPENG, XGEMM, CHECK_T2 and INITIN collisions fixed; bulk renaming via NEWLY_LINKED_MODULES; the audit tool; the Rundirmp2 and Runintgrt entry points; exclusions cleaned up.
2026-09-04 – 09-11	COMMON shadowing found (ABCD → the KS crash); the KS category brought the suite to 51/51; COMMON audit added; nine latent shadows renamed pre-emptively.
2026-10-14	Ported to a second machine (ccsd): init.F includes in column 1 (`73944a6d`); aces2 built with `-assume dummy_aliases` (`dbed9377` , ifx 2023.2 miscompiled vgamma2); sorted source lists, `LC_ALL=C` and extra renames (`eab3c4e6`). 51/51 and an identical 296-entry audit on both machines.
2026-10-23	Defects found while writing this manual fixed (`f97a9759`): `Hbar` → `Hbar_` in Runhbar/Buildhbar, `Iuhf` as `Integer*8` , Getreclenpy and Cgetrecpy return their values. Still 51/51 on both machines.
2026-10-23	TODO stage 1 (T-03, T-04, T-07, T-16): the `--calls` check fails the build on calls into COMMON blocks, unapplicable rename pairs stop the build, the extension suffix comes from `EXT_SUFFIX` , and the test roll-up and classifier were fixed.
2026-10-23	TODO stage 2 (T-11, T-12, T-15): the acespy legacy tree removed (tag `pyaces-legacy-20261023`), `Main` built against aces2's own headers (the old `lists.com` copy had a different `/lists/` layout), `Python/moio_len.py` made importable.
2026-10-23	TODO stage 3 (T-06, T-13, T-14) and T-02: f2py builds the module from the sources alone (`Main/aces2.pyf` removed), renames moved to `tools/localize.txt` , and the `python/pyaces` package with a subprocess `run()` .
2026-10-30	`classify_suite.py` reads the bare-name `test_results` entries upstream added (it stopped with a ValueError before), excludes `PROGRAM=PCCD` , and the regenerated TSV covers the 157-case aces2 suite: eight new CCSD cases bring the pyaces suite to 59/59.
2026-10-30	Runpccd fixed: it read ZMAT again after `Init()` and called `Ints_` with no arguments. It now runs single-point pCCD as xrunpccd does (no Vlambda/Pccd), with a new aces2 case `pccd_h2o_sto3g` (pyaces 60/60). Dropping those calls changed the link: `check_t2_` renames for hcmult and libc4, and the latent `symoff_` shadow is in the common-block baseline (`tools/localize.txt` , 21st instance).

2026-10-30	T-05: `Get1ehpy` returned zeros for every molecule. It took a negative `J0` from `Aces_malloc` (the heap lies below its anchor) for a failed allocation and returned before reading IIII; the irrep blocks were also placed at the wrong offsets. It now tests `Jcore(1)`, as `aces_malloc.c` says to, and builds the block-diagonal SO matrix. `Getlstpy` / `Putlstpy` check the list, the column range and `Ndim` (new `Getlstdimpy`); Python gets `list_shape` and `get_1e_ints`. Verified on H_2O 6-31G* CCSD in C_{2v} and C_1: `C^TSC = 1` to 5×10^{-15}, the same (H, S) eigenvalues and one-electron energy (to 10^{-11}), list 46 read irrep by irrep equal to `Getallpy`. pyaces 60/60.
2026-10-30	T-09: the C wrappers are compiled with `$PYACES_CC` (configure's C compiler, icx) instead of `mpicc`, and `-qopenmp` is gone from the Fortran flags. The .so has the same dependencies and no OpenMP symbols before or after; the 60 suite energies are unchanged.
2026-10-30	T-10: `pyaces/build.sh` run on its own follows `build_config.sh` (`aces_program_load_env`: modules, setvars or PATH, and configure's compilers) instead of loading the HiPerGator modules, which failed on a setvars host with a `module` command. The build log names the compilers. Run on its own, from `./build.sh` and with `PYACES_MODULES=""` it builds the same .so (audit 244, pyaces 60/60, energies unchanged).
2026-10-30	T-17: API tests in `pyaces/tests` (8 tests, records, Runscf RHF/UHF, two Runhbar branches, MO lists, one-electron integrals), run by `run_tests.sh pyaces` as the `py-api` row. They found two character-record defects, both fixed: `Cputrecpy` stored only 8 characters (`Character*8 Source`), and without `-DF_64BIT` `Cgetrecpy` asked for twice the length of a record written by aces2 and ended the process.

12.2 Deployment on the ccsd server

- Installed at `ycpark@172.24.7.130:/home/ycpark/program/ACES`. The server has no internet access, so it is updated from the workstation with `git push ccsd master` (the server repository has `receive.denyCurrentBranch=updateInstead`).
- Toolchain: `intel/oneapi_2023.2.0` (ifx 2023.2, MKL 2023.2, Intel MPI 2021.10) and `python/3.12.3`.
- Build and test only through PBS: `build_logs/build_ccsd.pbs` (`qsub -v TARGETS=pyaces …`), `build_logs/test_ccsd.pbs`, and `build_logs/rebuild_test.pbs` (clean aces2 rebuild plus tests).
- Built with `-march=core-avx2`, so the binaries run on any of the ccsd50/51/52 nodes.

12.3 Lessons from the toolchain differences

- After a compiler change, rerun all three suites (aces2, aces3, pyaces). ifx 2023.2 handled two pieces of code differently from 2024.0, which had passed them.
- Compare the link-audit results across machines. The same sources linked different implementations because of make and locale differences.

Appendix A. Legacy files

The acespy-era files were removed on 2026-10-23 (TODO T-11). To look at them, check out the tag `pyaces-legacy-20261023`:

```
git show pyaces-legacy-20261023:pyaces/Fxx/README
git checkout pyaces-legacy-20261023 -- pyaces/Fxx    # restore a directory into the working tree
```

Removed path	What it was
`Fxx/`	A copy of aces2 sources from the 2023 per-module shared-library build (45 directories, about 6,100 files, 45 MB), long drifted from aces2.
`include/`, `a2_pyf/include_no/`, `Main/*.com`	Old header copies. The `Main/*.com` copies were still read by init.F; their `lists.com` declared a smaller `/lists/` block (`maxref=1`, 0x61a80 bytes) than aces2's (0x64b540). init.F only includes the headers and never touches the variables, so nothing broke, but the layouts did not match.
`Main/Archive/`, `Main/compile_acespy`, `Main/GNUmakefile*`, `a2_pyf/GNUmakefile*`	Early prototypes and the old make / `f2py -c` build recipes.
`a2_pyf/Archive/`, `Main/Pilot_tests/`	Per-routine `.pyf` files and early COMMON/icore experiments.
`a2_pyf/*.f`, `Main/init.f`, `Main/aces2_main.f`	Stale preprocessed copies of the `.F` sources.
`Exp/`, `rename.py`	An f2py tutorial and a `.a` → `.so` renamer with a hard-coded path.
`suite_classification.tsv.before_*`	TSV snapshots taken before earlier classification fixes.

Still present: `README` and `frequently_asked_questions` (old notes and the generic f2py FAQ) and `LICENSE` (GPL v3, still in force).

Appendix B. Command summary

```
# Build (local workstation, inside the allocated cores)
taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./build.sh aces2 pyaces

# Tests
OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 \
  taskset -c 20-23 flock -s /tmp/qtoy-bench.lock ./run_tests.sh aces2 pyaces
cd pyaces && python3 pyaces_suite_runner.py 010a       # one case

# Link audit
cd pyaces && python3 tools/collision_audit.py
cd pyaces && python3 tools/collision_audit.py --update # only after verification

# Inspect the module
python3 -c "import sys; sys.path.insert(0,'pyaces/Main/build'); import aces2py; print(aces2
py.__doc__)"
nm pyaces/Main/build/aces2py*.so | grep ' runcc_$'

# Clean rebuild after changing aces2 flags
git clean -fdXq aces2 && ./build.sh aces2 pyaces

# ccsd server
git push ccsd master
ssh ycpark@172.24.7.130 'cd program/ACES/build_logs && qsub rebuild_test.pbs'
```